

CSC 4520/6520

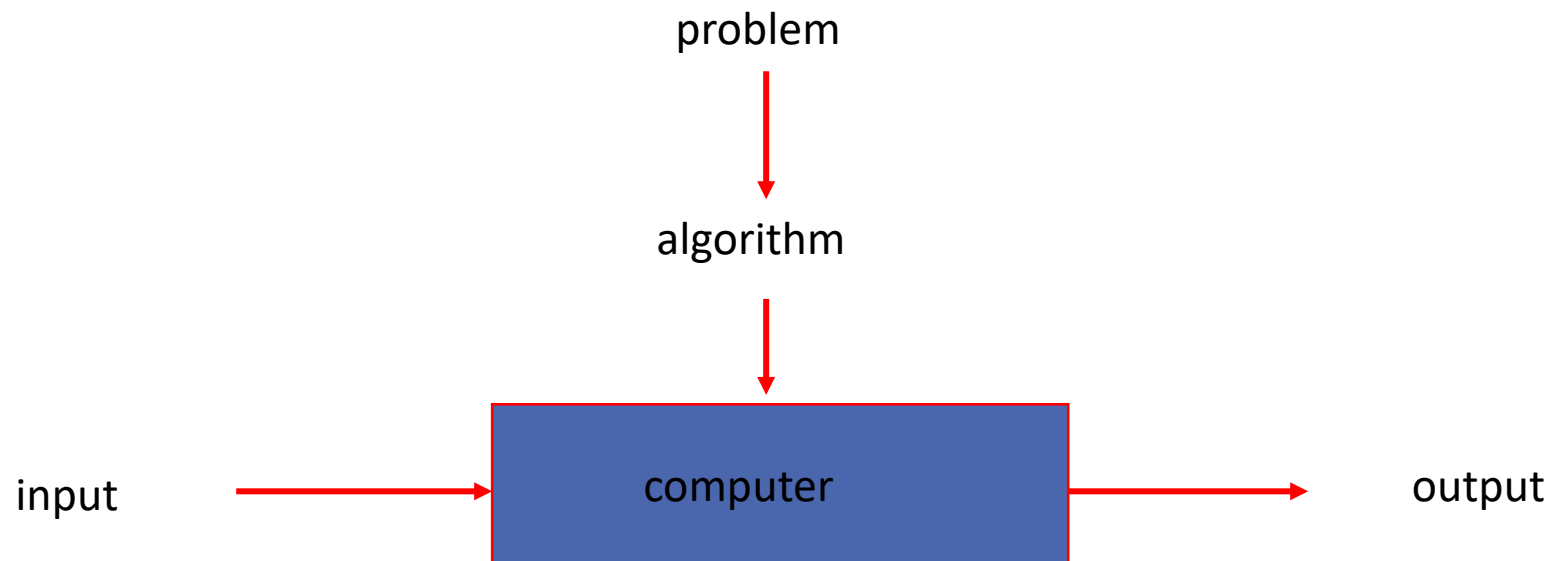
Design & Analysis of Algorithms

CONTENTS

ABDULLAH BAL, PHD

What is an algorithm?

- An algorithm is any well-defined **computational procedure** that takes some value, or set of values, as **input** and produces some value, or set of values, as **output** in a **finite amount of time**.
- An algorithm as **a tool** for solving a well-specified computational problem.



Two main issues related to algorithms

- ❑ How to **design** algorithms
- ❑ How to **analyze** algorithm efficiency

Analysis of Algorithms

- Analyzing an algorithm has come to mean predicting the resources that the algorithm requires.
 - Memory
 - Communication bandwidth
 - Energy consumption
 - Computational time

Analysis of Algorithms

ALGORITHM *NoName*($A[0..n - 1]$)

```
val  $\leftarrow A[0]$ 
for  $i \leftarrow 1$  to  $n - 1$  do
    if  $A[i] > \text{val}$ 
        val  $\leftarrow A[i]$ 
return val
```

Analysis of Algorithms

ALGORITHM *Sieve(n)*

//Implements the sieve of Eratosthenes

//Input: A positive integer $n > 1$

//Output: Array L of all prime numbers less than or equal to n

```
for  $p \leftarrow 2$  to  $n$  do  $A[p] \leftarrow p$ 
for  $p \leftarrow 2$  to  $\lfloor \sqrt{n} \rfloor$  do //see note before pseudocode
    if  $A[p] \neq 0$  //p hasn't been eliminated on previous passes
         $j \leftarrow p * p$ 
        while  $j \leq n$  do
             $A[j] \leftarrow 0$  //mark element as eliminated
             $j \leftarrow j + p$ 
//copy the remaining elements of  $A$  to array  $L$  of the primes
 $i \leftarrow 0$ 
for  $p \leftarrow 2$  to  $n$  do
    if  $A[p] \neq 0$ 
         $L[i] \leftarrow A[p]$ 
         $i \leftarrow i + 1$ 
return  $L$ 
```

Analysis of Algorithms

ALGORITHM $F(n)$

//Computes the n th Fibonacci number recursively by using its definition

//Input: A nonnegative integer n

//Output: The n th Fibonacci number

if $n \leq 1$ **return** n

else return $F(n - 1) + F(n - 2)$

Analysis of Algorithms

- The Worst-case analysis
- The Best-case analysis
- The Average-case analysis
- Asymptotic analysis
- Order of Growth

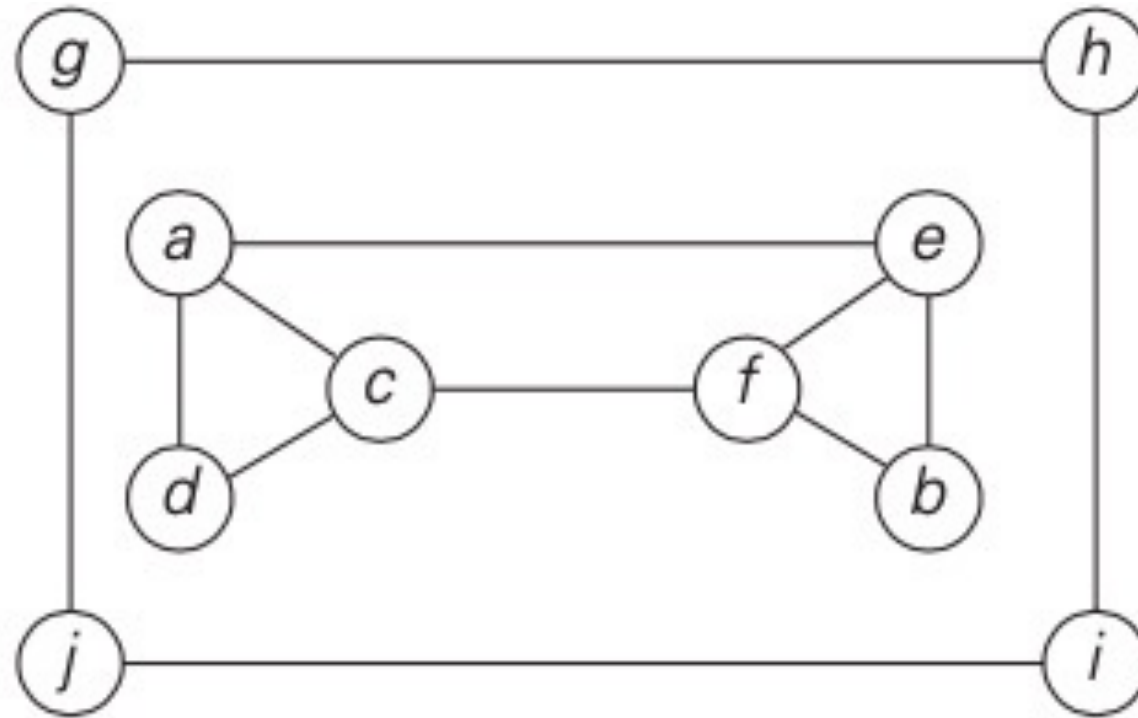
Algorithm design techniques/strategies

Sorting problem:

7 5 8 2 3 9 11 10 1 3 2 9 4 6 5 99 88 11 22 45 65

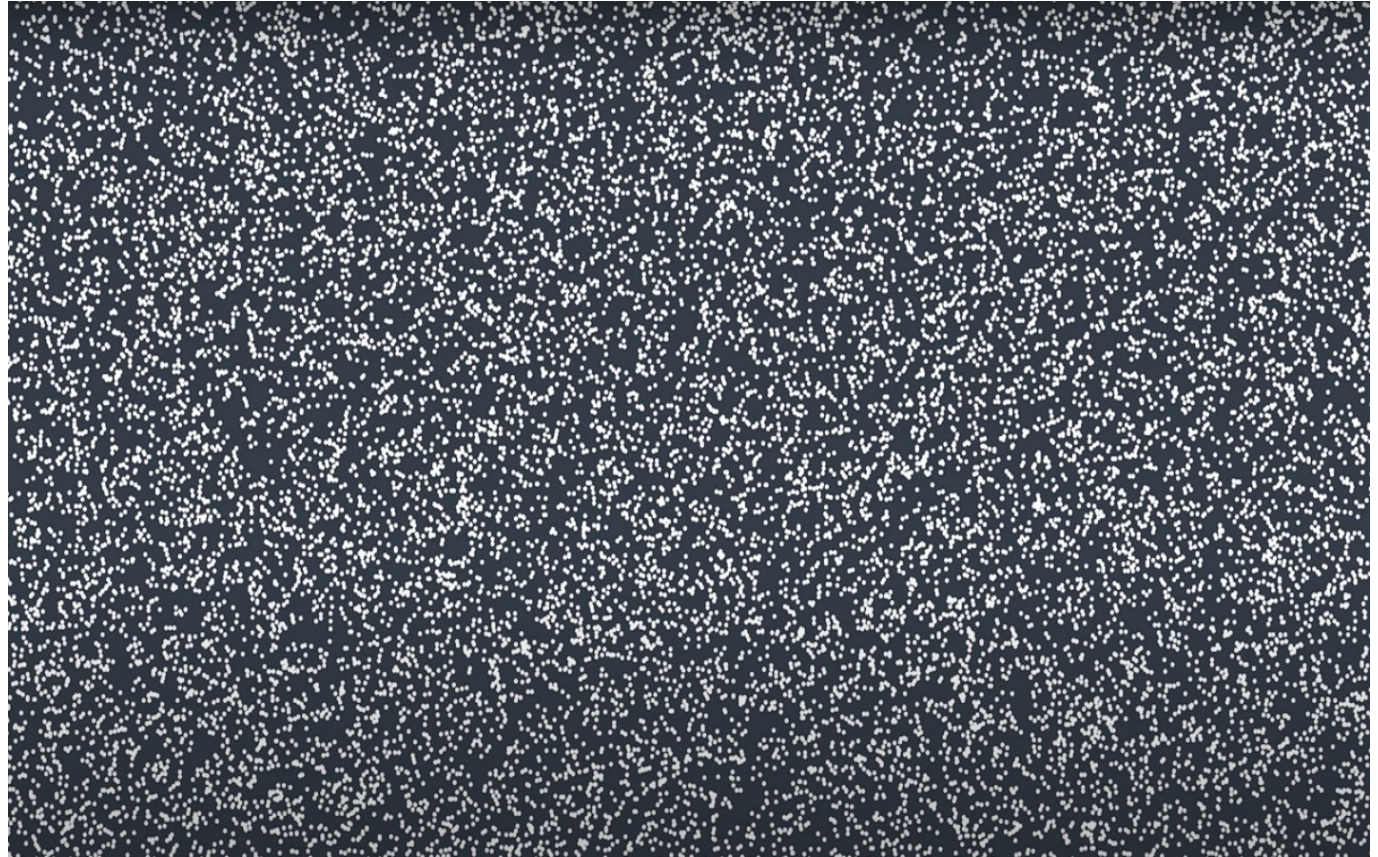
Algorithm design techniques/strategies

Graph problems:



Algorithm design techniques/strategies

Closest pair problem:



Important problem types

- **Sorting** (Insertion, Merge, Quick, Bubble, Selection, Counting, Heap...)
- **Searching** (Binary, Linear...)
- **String Processing** (Matching, splitting)
- **Graph Problems** (Shortest path, transitive closure, minimum spanning trees, social networks)
- **Combinatorial Problems** (Generating permutation, combinations)
- **Geometric Problems** (The closest-pair problem)
- **Numerical Problems** (Matrix multiplication, GCD, LCM)

Algorithm design techniques/strategies

Brute force

- Bubble sort
- Selection sort
- Closest pair problems
- String matching
- Travelling Salesman problems
- Knapsack problem
- Graph Traversal

Algorithm design techniques/strategies

Decrease and conquer

- Insertion sort
- Topological sorting
- Algorithms for generating permutations, subsets

Algorithm design techniques/strategies (Cont.)

Divide and conquer

- Merge sort
- Quick sort
- The closest pair problem

Algorithm design techniques/strategies (Cont.)

Transform and conquer

- Balanced trees (AVL, 2-3 trees)
- Heaps and Heapsort

Algorithm design techniques/strategies

Space and time tradeoffs

- Counting sort
- Hashing
- String matching

Algorithm design techniques/strategies

Dynamic programming

- Optimal solutions
- Efficient recursion
- The shortest path problems

Algorithm design techniques/strategies

Greedy technique

- Tree problems (MST)
- Huffman trees and codes

Data Structures

- List
 - Array
 - Linked List
 - String
- Stack
- Queue
- Priority Queue
- Graph
- Tree
- Set and Dictionary

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.